Funkcje

Funkcja to wydzielona część programu wykonująca określone operacje, możliwa do użycia podczas wykonywania programu.

Funkcje stosuje się, aby uprościć program główny i zwiększyć czytelność kodu.

```
print()

input()

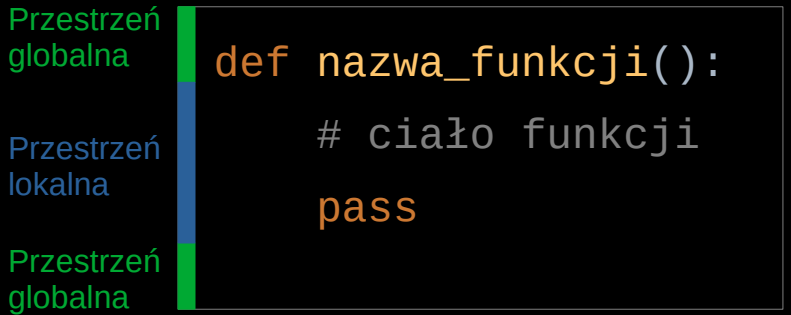
int()

float()

type()
```

Definicja Funkcji

Funkcje należy zdefiniować i robi się to za pomocą słowa def, następnie podaje się nazwę funkcji i parę nawiasów okrągłych,
linijka z definicją funkcji zakończona jest dwukropkiem. Ciało funkcji definiuje się w następnych linijkach z uwzględnieniem wcięć



Definicja Funkcji

Przestrzeń globalna

Przestrzeń lokalna

Przestrzeń globalna

```
def wypiszTest():
    print("test")

wypiszTest()
```

```
test
```

```
def komunikat():
    print("Cześć!")

komunikat()
komunikat()
```

```
Cześć!
Cześć!
```

Działanie Funkcji

Wywołanie funkcji w kodzie przerywa dalsze wykonywanie kodu w tym miejscu. Przechodzi do ciała funkcji i ją wykonuje. Po zakończeniu wykonywania funkcji wraca do miejsca wywołania funkcji i kontynuuje działanie programu.

Przestrzeń globalna

Przestrzeń lokalna

Przestrzeń globalna

```
def komunikat():
    print("Uwaga! Komunikat!")

print("Witaj!")
komunikat()
print("Żegnaj")
```

```
Witaj!
Uwaga! Komunikat!
Żegnaj
```

Parametry Funkcji

Funkcja może być napisana w dowolnym miejscu w kodzie. Wywołanie funkcji musi nastąpić po jej definicji. Nie wolno nazywać zmiennej tak samo jak funkcji.

Parametry służą do przekazywania danych do wewnątrz funkcji.

Parametry dostępne są wewnątrz funkcji.

```
def wypisz(parametr):
    # ciało funkcji
    # dostęp do parametrów
    print(parametr)

wypisz("Hej!")
```

```
Hej!
```

Parametry Funkcji

```
def hej(imie):
    print("Hej " + imie + "!")

hej("Adam")
```

```
Hej Adam!
```

```
def dodaj(liczba):
    print("Wynik: ", liczba + liczba)

dodaj(5)
```

```
Wynik:  10
```

Wiele parametrów

Funkcje mogą przyjmować wiele parametrów.

```
def nazwa_funkcji(parametr1, parametr2):
    # ciało funkcji
    # dostęp
```

```
def hej(imie1, imie2):
    print("Hej " + imie1 + " i " + imie2 + "!")

hej("Adam", "Anna")
```

```
Hej Adam i Anna!
```

Parametry pozycyjne Funkcji

Wywołując funkcję z argumentami wpisanymi po kolei używamy pozycyjnego przekazania argumentów.

```
def powitaj(imie, nazwisko):
    print("Dzień dobry: " + imie + " " + nazwisko + "!")

powitaj("Adam", "Kowalski")
```

```
Dzień dobry: Adam Kowalski!
```

Parametry z wykorzystaniem słów kluczowych

Możemy wywołać funkcję z argumentami używając nazwa parametrów zamiast wpisywać je w odpowiedniej kolejności.

```
def powitaj(imie, nazwisko):
    print("Dzień dobry: " + imie + " " + nazwisko + "!")

powitaj(nazwisko = "Kowalski", imie = "Adam")
```

```
Dzień dobry: Adam Kowalski!
```

Parametry domyślne Funkcji

Możemy definiować argumenty w funkcji wraz z domyślnymi wartościami.

Parametry, które są domyślne muszą być na końcu na liście parametrów.

```
def powitaj(imie, nazwisko=""):
    print("Dzień dobry: " + imie + " " + nazwisko + "!")
powitaj(imie = "Adam")
```

```
Dzień dobry: Adam !
```

```
def powitaj(imie, nazwisko="Kowalski"):
    print("Dzień dobry: " + imie + " " + nazwisko + "!")
powitaj(imie = "Adam")
```

```
Dzień dobry: Adam Kowalski!
```

Zwracanie wyniku z Funkcji

Każda funkcja zwraca jakąś wartość. Do zwracania wartości z funkcji służy instrukcja return. Jeżeli funkcja nie zwracamy wartości, to domyślnie zwracana jest wartość None.

```
def nazwa_funkcji():
    return wartosc
```

```
def powitaj(imie, nazwisko):
    return "Dzień dobry: " + imie + " " + nazwisko + "!"

print(powitaj("Adam", "Kowalski"))
```

```
Dzień dobry: Adam Kowalski!
```

Wartość None

- Jest to wartość, która nie ma żadnej wartości.
- Jest to słowo kluczowe.
- Nie stanowi on żadnych danych i jest bytem domyślnie zwracanym z funkcji które nie mają instrukcji return.
- None może być przypisane do zmiennej i może być używane do porównywania.

```
def test():
    pass

if(test() == None):
    print("None!")
```

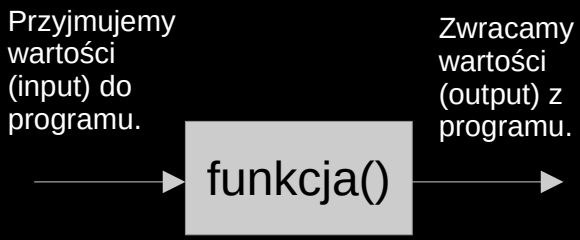
```
None!
```

Listy jako argumenty Funkcji

Funkcja jako argumenty może oprócz literałów przyjmować również typy bardziej złożone, np. listy.

```
def suma(lista):
    suma = 0
    for i in lista:
        suma += i
    return suma
print(suma([1, 2, 3, 5]))
```

```
11
```



Ćwiczenie

Eksperymenty z funkcjami

ćwiczenie 15 minutowe

- Zmodyfikuj kalkulator tak aby używał funkcji do obliczania odpowiednich działań arytmetycznych.
- Napisz funkcję która sprawdzi czy liczba jest pierwsza i zwróci wartość True lub False.

Ćwiczenie

Eksperymenty z funkcjami

ćwiczenie 40 minutowe

- Napisz program z wykorzystaniem funkcji który jako parametr przyjmie rok i zwróci informację czy podany rok jest przestępny czy nie.
- Dodaj funkcję która jako parametr przyjmuje rok i miesiąc, a jako wynik zwraca informację ile dni jest w danym miesiącu.
- Napisz funkcję która przyjmuje trzy parametry - rok, miesiąc i dzień, sprawdzi czy wprowadzone dane są poprawne, jeżeli nie zwróci wartość None, jeżeli są poprawne zwróci informację który to dzień roku.

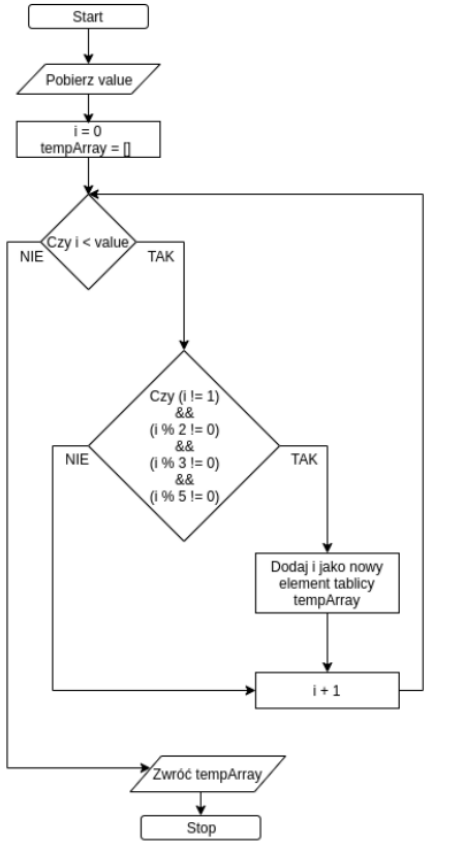
Ćwiczenie

Eksperymenty z funkcjami

ćwiczenie 30 minutowe

- Wzorując się na schemacie blokowym, proszę napisać program, który odflitruje z danego przedziału wszystkie liczby pierwsze. Wartość X i Y, należy odebrać od użytkownika.

Rysunek 1: Schemat blokowy algorytmu



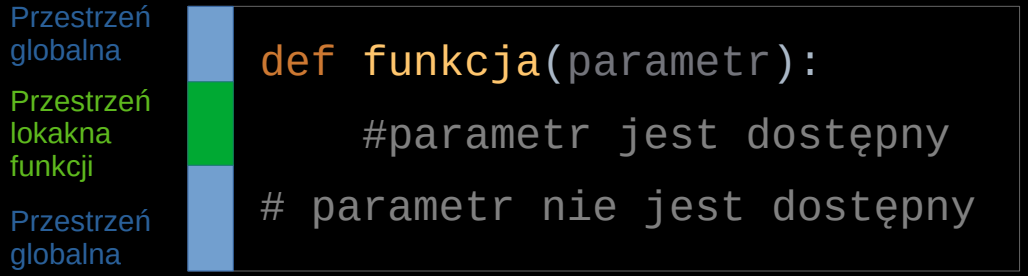
*tablica = lista

Algorytm Sito Eratostenesa

Algorytm ten jest przypisywany Eratostenesowi z Cyreny i służy do wyznaczania liczb pierwszych. Algorytm jest zbudowany w taki sposób aby wykluczyć kolejno liczby, które nie są liczbami pierwszymi. Aby odszukać liczby pierwsze mniejsze od danej liczby value na podanym schemacie należy odsiać liczbę 1 oraz wszystkie liczby złożone, czyli posiadające więcej niż dwa dzielniki. Kolejnymi krokami wykluczane są liczby podzielne przez: 2, 3 oraz 5. W następstwie użycia algorytmu zostaje wyznaczona tablica liczb pierwszych. Liczby pierwsze to takie liczby naturalne większe od 1, które mają dokładnie dwa dzielniki naturalne: jedynkę i siebie samą.

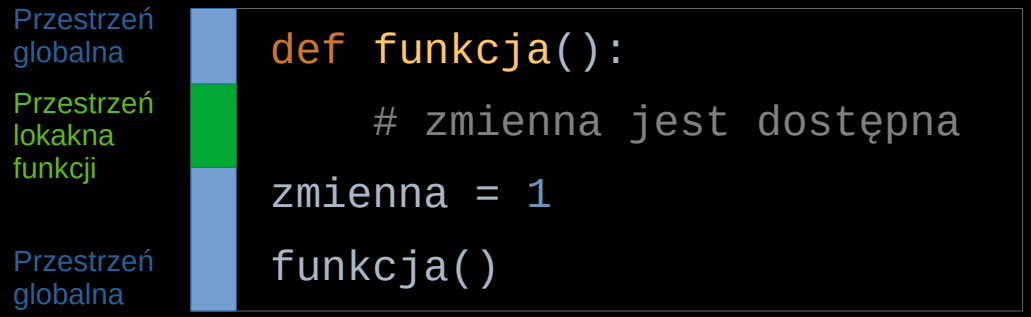
Funkcje i zakresy

Zakresem nazwy (ang. name scope), np. nazwy zmiennej, jest ta część kodu, w której nazwa jest w pełni rozpoznawalna. Np. parametr funkcji jest widoczny tylko w danej funkcji, więc jego zakresem jest cała funkcja.



Funkcje i zakresy

Zmienna znajdująca się poza zakresem funkcji jest widoczna w tej funkcji. Jednak dostępna jest tylko do odczytu, nie ma możliwości jej modyfikacji próba modyfikacja tworzy lokalną zmienną.




```
def funkcja():
    zmienna = 5
    print(zmienna)
zmienna = 1
funkcja()
print(zmienna)
```

```
5
1
```

Funkcje i zakresy

Zmienna znajdująca się poza zakresem funkcji jest widoczna w tej funkcji. Jednak dostępna jest tylko do odczytu, nie ma możliwości jej modyfikacji próba modyfikacja tworzy lokalną zmienną.

```
def funkcja():
    print(zmienna)
zmienna = 1
funkcja()
print(zmienna)
```

```
1
1
```

Funkcje i zakresy

```
def funkcja():
    zmienna = 2
    print(zmienna) # 2
zmienna = 1
funkcja()
print(zmienna) # 1
```

```
2
1
```

Funkcje - słowo kluczowe global

Istnieje możliwość edytowania zmiennych spoza zakresu, służy do tego słowo kluczowe global

```
def funkcja():
    global zmienna
    zmienna = 5
    print(zmienna)
zmienna = 4
funkcja()
print(zmienna)
```

```
5
5
```

Ćwiczenie

Eksperymenty z użyciem funkcji print()

ćwiczenie 5 minutowe

- Sprawdź jak działa zakres w języku Python.
- Napisz funkcję obliczającą silnie, funkcja przyjmuje liczbę całkowitą i oblicza silnię dla niej.

$$3! = 1 \cdot 2 \cdot 3 = 6$$

$$4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$$

Rekurencja

Rekurencja, zwana także rekursją (ang. recursion, z łac. recurrere, przybiec z powrotem) - odwoływanie się np. funkcji do samej siebie.

```
def Dodaj(argument):
    if argument > 5:
        return 0
    else:
        return argument + Dodaj(argument+1)# 1 + 2 + 3 + 4 + 5

print(Dodaj(1))
```

```
15
```

Ćwiczenie

Eksperymenty z użyciem funkcji rekurencyjnej

ćwiczenie 45 minutowe

- Napisz funkcję obliczającą silnie, funkcja przyjmuje liczbę całkowitą i oblicza silnię dla niej – **wykorzystaj rekurencję**.
- Zmodyfikuj poprzedni program aby liczył silnie za pomocą rekurencji wykorzystując rekurencję oblicz ciąg fibonacciego.

$$3! = 1 \cdot 2 \cdot 3 = 6$$

$$4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$$

Ciąg Fibonacciego – ciąg liczb naturalnych określony rekurencyjnie w sposób następujący:

Pierwszy wyraz jest równy 0, drugi jest równy 1, każdy następny jest sumą dwóch poprzednich.

Formalnie:

$$F_n := \begin{cases} 0 & \text{dla } n = 0, \\ 1 & \text{dla } n = 1, \\ F_{n-1} + F_{n-2} & \text{dla } n > 1. \end{cases}$$

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181

Typy sekwencyjne

Typ sekwencyjny jest typem danych języka Python, który może przechowywać więcej niż jedną wartość (lub mniej niż jedną, ponieważ dana sekwencja może być pusta) a wartości te mogą być przeglądane sekwencyjnie (stąd nazwa), element po elemencie. Pętla for jest przeznaczona do iterowania po sekwencyjnych typach.

Mutowalność

Mutowalność jest własnością typu danych która umożliwia modyfikowanie tego typu w czasie jej istnienia. W języku Python mamy dane mutowalne i Niemutowalne.

- Dane mutowalne to takie, które możemy dowolnie modyfikować w trakcie działania programu, np. liczby całkowite, ciągi znaków, listy.
- Dane niemutowalne to takie które możemy zainicjować i odczytać, nie możemy ich modyfikować po inicjalizacji.

Krotka

Krotki (tuples) to niemutowalny typ sekwencyjny. Krotki mogą zachowywać się jak lista, ale nie mogą być modyfikowane po inicjalizacji.

Krotki mogą przechowywać dowolne typy danych. Krotki tworzymy za pomocą nawiasów okrągłych ().

```
krotka1 = ()
krotka2 = (1, 2)
krotka3 = (1, 2, 3, 4)
print(krotka1, krotka2, krotka3)
```

```
() (1, 2) (1, 2, 3, 4)
```

Krotka

Dane zapisane w Tuple mogą być różnego typu. Tuple jest zbiorem, którego nie można edytować (brak możliwości usuwania, dodawania i edycji elementów). Indeksy w tuple rozpoczynają się od 0.

```
krotka = ("tekst", True, 25)
```

```
krotka = ("tekst", True, 25)

print(krotka)

print(krotka[0])
print(krotka[2])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
('tekst', True, 25)
tekst
25
```

Dozwolone operacje na krotkach

Krotek nie można modyfikować po ich utworzeniu, jednak można na nich wykonywać inne operacje, tak jak na listach:

- **len()** - odczytanie ilości elementów w krotce.
- **+** - operacja dodawania dwóch krotek do siebie.
- ***** - operacja mnożenia krotek (powielania).
- **in, not in** - sprawdzenie czy wartość istnieje w krotce.

Dozwolone operacje na krotkach

```python
krotka = (45, 5, 3, 4.5, "tekst", True)
print(krotka)
print(len(krotka))
print(krotka + krotka)
print(krotka * 2)
print(45 in krotka, 45 not in krotka)
```

```
(45, 5, 3, 4.5, 'tekst', True)
6
(45, 5, 3, 4.5, 'tekst', True, 45, 5, 3, 4.5, 'tekst', True)
(45, 5, 3, 4.5, 'tekst', True, 45, 5, 3, 4.5, 'tekst', True)
True False
```

Krotki - przykłady

```python
krotka = ("tekst", True, 25, 25, 36)
wyciete_z_krotki = krotka[2:]

print(wyciete_z_krotki)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
(25, 25, 36)
```

```python
krotka = ("tekst", True, 25, 25, 36)

for n in krotka:
    print(n)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
tekst
True
25
25
36
```

Ćwiczenie

Eksperymenty z użyciem krotek

Ćwiczenie 15 minutowe

- Sprawdź jak krotki zachowują się w działaniu, wykorzystaj poznane sposoby działania na krotkach.

Słownik

Słownik to typ danych, służący do przechowywania danych w sposób słownikowy, tzn. wartości słownika nie muszą być indeksowane jak to miało miejsce w przypadku list i krotek, ale zawierają klucze identyfikujące wartość.
Słownik nie jest to typ sekwencyjny (ale można go z łatwością zaadaptować do przetwarzania sekwencyjnego) i jest mutowalny.

- Słownik to zestawienie par klucz-wartość.
- Każdy klucz musi być unikalny.
- Klucz może być danymi dowolnego typu.
- Funkcja len() działa w słownikach.

Słownik

Dane zapisane w Słowniku mogą być różnego typu. Słownik jest zbiorem, którym można w łatwy sposób manipulować (dodawanie, usuwanie, edytowanie jej wartości). Nie posiada indeksów. Składa się z par „klucz” i „wartość”.

```
slownik = {"Imie": "Adam", "Nazwisko": "Kowalski", "Wiek": 18}
```

Klucz
Klucz musi być unikalny.

wartość

```
slownik = {"Imie": "Adam",
           "Nazwisko": "Kowalski",
           "Wiek": 18,
           "Zainteresowania": ["Gra w piłkę", "Gra w tenisa"]}

print(slownik["Imie"])
print(slownik["Zainteresowania"])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
Adam
['Gra w piłkę', 'Gra w tenisa']
```

Tworzenie słownika

Słowniki tworzymy za pomocą nawiasów klamrowych {}

```
slownik = {}
slownik = {"imie": "Jan", "nazwisko": "Kowal"}
listaNumerow = {"Adam": 555865500, "Adela": 508436222}

print(slownik)
print(listaNumerow)
```

```
{'imie': 'Jan', 'nazwisko': 'Kowal'}
{'Adam': 555865500, 'Adela': 508436222}
```

Słownik

```
slownik = {"Adam": 555865500, "Adela": 508436222}

print(slownik)
print(slownik["Adam"])
print(slownik["adam"])#rozrózniana jest wielkosc liter!
```

```
{'Adam': 555865500, 'Adela': 508436222}
555865500
```

```
    print(slownik["adam"])#rozrózniana jest wielkosc liter!
KeyError: 'adam'
```

Słownik

```python
slownik = {"Imie": "Adam", "Nazwisko": "Kowal"}


print("Adam" in slownik)
print("Adam" not in slownik)
print("Nowak" in slownik)
print("Nowak" not in slownik)
```

```
False
True
False
True
```

Iterowanie po elementach słownika

Słowniki nie są sekwencyjnym typem danych, czyli nie można na nich użyć pętli for. Ale możemy wyciągnąć listę kluczy z naszego słownika i na niej wykonać pętlę for, służy do tego metoda keys()

```
slownik = {"Imie": "Adam", "Nazwisko": "Kowal"}
for k in slownik.keys():
    print(k)

for v in slownik.values():
    print(v)

for k, v in slownik.items():
    print(k, " : ", v)
```

```
Imie
Nazwisko
Adam
Kowal
Imie  :  Adam
Nazwisko  :  Kowal
```

Metody słowniki

- **Metoda keys()** umożliwia wyciągnięcie listy kluczy z naszego słownika i na podstawie jej wykonać pętlę for.
- **Metoda values()** zwraca listę wszystkich wartości w słowniku.
- **Metoda items()** zwraca listę krotek w której pierwszy element to klucz a drugi to wartość umieszczona pod tym kluczem.

```
for k in slownik.keys():
    for v in slownik.values():
        print(k, v)
        break
```

```
Imie Adam
Nazwisko Adam
```

Słowniki - modyfikacja danych

Dane w słownikach modyfikować można przez odwołanie się do elementu słownika poprzez klucz i przypisanie do niego nowej wartości.

```
slownik = {"Imie": "Adam", "Nazwisko": "Kowal"}

slownik["Telefon"] = 565_555_555
slownik["Imie"] = "Amadeusz"
print(slownik)
```

```
{'Imie': 'Amadeusz', 'Nazwisko': 'Kowal', 'Telefon': 565555555}
```

Dodawanie wartości do słownika

Dane do słownika dodajemy poprzez przypisanie wartości do klucza, który w słowniku do tej pory nie istniał alternatywnie a użyć metody **update()**

```
slownik = {"Imie": "Adam", "Nazwisko": "Kowal"}
slownik["Imie"] = "Amadeusz"
print(slownik)

slownik.update({"Telefon": 555_555_555})
print(slownik)

slownik.update({"Telefon": 666_666_666})
print(slownik)
```

```
{'Imie': 'Amadeusz', 'Nazwisko': 'Kowal'}
{'Imie': 'Amadeusz', 'Nazwisko': 'Kowal', 'Telefon': 555555555}
{'Imie': 'Amadeusz', 'Nazwisko': 'Kowal', 'Telefon': 666666666}
```

Usuwanie elementów słownika

Usuwanie elementów ze słownika odbywa się za pomocą instrukcji **del** z podaniem klucza który chcemy usunąć alternatywnie używając metody **popitem()** usuwa ostatni element.

```
slownik = {"Imie": "Adam", "Nazwisko": "Kowal", "Telefon" : 555_555_555}
slownik.popitem()
print(slownik)

del(slownik["Nazwisko"])
print(slownik)
```

```
{'Imie': 'Adam', 'Nazwisko': 'Kowal'}
{'Imie': 'Adam'}
```

Dodawanie, modyfikowanie, usuwanie słowniki

W słownikach możemy wykorzystywać funkcję **len()**

Usuwanie elementów można wykonać używając:

- Słowa kluczowego del `del slownik["Imie"]`
- Metody popitem() usuwającej ostatni element `slownik.popitem()`

Dodawanie elementów można wykonać za pomocą:

- Przypisania wartości do klucza `slownik["Imie"] = "Amadeusz"`
- Wykorzystania metody update() `slownik.update({"Telefon": 555_555_555})`

Modyfikowanie elementów można wykonać za pomocą:

- Przypisania wartości do klucza `slownik["Imie"] = "Amadeusz"`
- Wykorzystania metody update() `slownik.update({"Telefon": 555_555_555})`

Słownik

```
slownik = {"Adam": 555865500, "Adela": 508436222}
del(slownik["Adam"])
print(slownik)#{'Adela': 508436222}

slownik["Adam"] = 555865500
print(slownik)#{'Adela': 508436222, 'Adam': 555865500}

slownik.update({"Adam": 55555555, "Damian": 556665566})
print(slownik)#{'Adela': 508436222, 'Adam': 55555555, 'Damian': 556665566}
```

```
{'Adela': 508436222}
{'Adela': 508436222, 'Adam': 555865500}
{'Adela': 508436222, 'Adam': 55555555, 'Damian': 556665566}
```

Słowniki – przykłady.

```python
slownik = {"Imie": "Adam", "Nazwisko": "Kowalski"}

slownik["Imie"] = "Patryk"
print(slownik["Imie"])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
Patryk
```

```python
slownik = {"Imie": "Adam", "Nazwisko": "Kowalski"}

slownik.update({"Miasto": "Gdynia"})
print(slownik)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
{'Imie': 'Adam', 'Nazwisko': 'Kowalski', 'Miasto': 'Gdynia'}
```

```python
slownik = {"Imie": "Adam", "Nazwisko": "Kowalski"}

del slownik["Imie"]
print(slownik)
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
{'Nazwisko': 'Kowalski'}
```

```python
slownik = {"Imie": "Adam", "Nazwisko": "Kowalski"}

for key in slownik:
    print(slownik[key])
```

```
(venv) krylew@krylew:~/PycharmProjects/untitled1$ python testdoprez.py
Adam
Kowalski
```

Unikalny zbiór - set

Set to kolekcja unikalnych wartości. Moduł sets udostępnia klasy służące tworzeniu nieuporządkowanych kolekcji unikatowych elementów oraz wykonywaniu na nich różnego rodzaju operacji.

```
zbior = {3, 4, 2}
zbior.add(3)
print(zbior)
```

```
{2, 3, 4}
```

```
zbior = set({3, 4, 2})
zbior.add(3)
print(zbior)
```

```
{2, 3, 4}
```

Sprawdzanie różnic w zbiorach set

```
zbior = {3, 4, 2}
zbior2 = {3, 4, 2, 6, 7, 8}
print(zbior.difference(zbior2))
print(zbior2.difference(zbior))
```

```
set()
{8, 6, 7}
```

```
zbior = {3, 4, 2}
print(3 in zbior)
print(3 not in zbior)
print(5 not in zbior)
```

```
True
False
True
```

Ćwiczenie

Eksperymenty z użyciem słowników

ćwiczenie 15 minutowe

- Sprawdź jak słowniki zachowują się w działaniu, wykorzystaj poznane metody.
- Napisz program który poprosi o wprowadzenie imienia ucznia oraz oceny jakie uzyskał, "x" kończy wprowadzanie danych, po zakończeniu wprowadzania danych zwraca średnią ocen dla każdego ucznia oraz dla całej klasy.

Na przykład średnią czterech liczb, –5, –3, 0 i 12, jest

$$\frac{-5+(-3)+0+12}{4}=1.$$

Moduły w Python

Moduł to plik z rozszerzeniem .py zawierający definicje i instrukcje Pythona, które można później zaimportować i użyć. Moduł identyfikowany jest przez nazwe.

Moduły importujemy do swojego programu za pomocą komendy import.

Elementy modułu mogą być

- Funkcjami.
- Zmiennymi.
- Stałymi.
- Klasami.
- Obiektami.

```
import math
import sys
import math, sys
```

Moduły - przestrzeń nazw (namespace)

Wszystkie moduły (pliki) mają swoją własną przestrzeń nazw. Aby skorzystać z elementów z zaimportowanego modułu należy użyć jego nazwy oraz znaku `.` (kropka) i elementu który chcemy użyć.

```
import math

print(math.sqrt(4))
print(math.pow(4, 3))#4 do potęgi 3
print(math.sin(math.pow(2, 2)))
```

```
2.0
64.0
-0.7568024953079282
```

Moduły - Importowanie pojedynczych elementów

Możemy importować moduły w całości, dodatkowo Python umożliwia importowanie pojedynczych elementów danego modułu.

Aby zaimportować pojedyńczy element wykorzystamy instrukcję:

from nazwa_modulu import pojedynczy_element

```
from math import pow, pi

print(pow(4, 3))#4 do potęgi 3
print(pow(pi, 2))
```

```
64.0
9.869604401089358
```

Importowanie całego modułu

Python umożliwia zaimportowanie wszystkich elementów z modułu.

```
from math import *

print(pow(4, 3))#4 do potęgi 3
print(pow(pi, 2))
```

```
64.0
9.869604401089358
```

Alias importowanego modułu

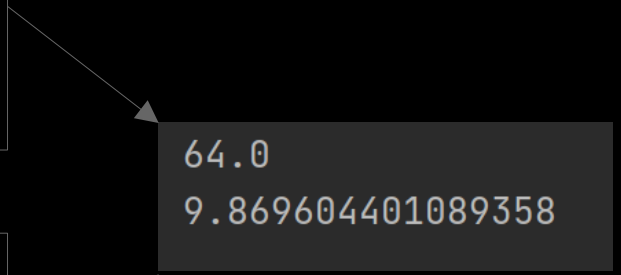
Python umożliwia zaimportowanie modułu wraz z nadaniem mu nowej nazwy.

```
import math as matma

print(matma.pow(4, 3))#4 do potęgi 3
print(matma.pow(matma.pi, 2))
```

```
from math import pow as potega, pi as PI

print(pow(4, 3))#4 do potęgi 3
print(pow(PI, 2))
```



Przykład - tworzenie własnego modułu

Główny plik projektu:

```
import Modul_Suma

print(Modul_Suma.Suma(2, 5))
```

```
7
```

Plik Modul_Suma.py

```
def Suma(a, b):
    return a + b
```

Ćwiczenie

Eksperymenty z użyciem modułów

ćwiczenie 20 minutowe

- Stwórz przykładowe moduły i zaimportuj elementy pomiędzy nimi.
- zmodyfikuj swoje dotychczasowe programy (np. kalkulator), tak aby ich kod rozdzielony był na poszczególne moduły.

Moduł math

```
cos()
pi
e
exp(x) # e^x
log(x) # logarytm naturalny z x
log(x, y) # logarytm z x przy podstawie y
ceil(x) # zaokrąglenie do wyższej liczby x
sqrt(x) # pierwiastek z x
floor(x) # zaokrąglenie do niższej liczby x
factorial(x) # x!
```

```
from math import sin
a = sin()
```

```
import math
a = math.sin()
```

Moduł – sprawdzanie elementów modułu

Python daje możliwość sprawdzenia w kodzie wszystkich elementów dostarczanych przez moduł można wykorzystać do tego funkcję dir().

```
import math

print(dir(math))
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc',
'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf',
'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf',
'nan', 'pi', 'pow', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
```

Funkcja random do losowania liczb

```
from random import random, randrange, randint

random()
# liczby całkowite
print(randrange(5)) #5 to górny zakres
print(randrange(1, 5)) #1 to dolny a 5 to górny zakres
print(randrange(1, 8, 2)) #1 to dolny a 8 to górny zakres, skok o 2
print(randint(1, 5)) #1 to dolny a 5 to górny zakres
```

```
3
3
3
5
```

Parametry Funkcji

Liczby zwracane przez funkcje losujące `randrange` i `randint` dane są tylko z pozornie losowe.

Funkcje te zwracają losowe liczby są w rzeczywistości algorytmami, które na podstawie przekazanego argumentu (ziarna) zwracają kolejne wartości.

Takie "Ziarno" jesteśmy w stanie ustawić za pomocą funkcji seed()

```
from random import randint, seed
#seed() # ustawienie ziarna jako aktualny czas
for i in range(10):
    print(randint(0, 10))

print("-----------Separator")
seed(255)
for i in range(10):
    print(randint(0, 10))
```

```
7
5
10
5
10
3
8
7
1
9
-----------Separator
0
5
10
10
9
0
5
9
2
8
```

Parametry Funkcji

Python pozwala losować wartość ze wskazanego zbioru wykorzystuje się do tego funkcje choice() i sample().

```
from random import choice, sample
listaImion = ["Anna", "Tomasz", "Iwona", "Augustyn"]
for i in range(7):
    print(choice(listaImion))

print(sample(listaImion, 3)) # sample(zbior, ilosc_elementow)
```

```
Augustyn
Augustyn
Augustyn
Iwona
Augustyn
Augustyn
Iwona
['Augustyn', 'Tomasz', 'Iwona']
```

Ćwiczenie

Eksperymenty z użyciem słowników

ćwiczenie 15 minutowe

Proszę napisać program który:

- Przyjmie od użytkownika 10 wartości i zapisze je w liście. Następnie proszę wylosować z tej listy 2 wartości.

Łańcuchy znaków

Programy komputerowe przechowują litery (znaki) jako liczby.
Każda litera (znak) ma przypisany sobie numer kodowany przy użyciu określonego standardu kodowania znaków. Najpopularniejszy z nich to ASCII (skrót od American Standard Code for Information Interchange).
Kod ASCII może przechowywać 256 znaków, pierwsze 128 to wszystkie najpopularniejsze znaki i litery alfabetu łacińskiego. Punkt kodowy to nic innego jak liczba tworząca znak.Kod ASCII wykorzystuje pierwsze 128 znaków z możliwych 256, pozostałe 128 jest wykorzystywane w zależności od danego języka lub dialektu np. strona kodowa ISO/IEC 8859-2 dla języków słowiańskich.

Łańcuchy znaków

Unicode jesr standardem, za pomocą którego możemy zapisać ponad milion znaków. Pierwsze 128 punktów kodowych Unicode jest identycznych w stosunku do ASCII, a pierwsze 256 punktów kodowych Unicode jest identycznych ze stroną kodową ISO/IEC 8859-1 (stroną kodową zaprojektowaną dla języków Europy Zachodniej). Unicode nie mówi nic natomiast o tym, jak dane znaki mają być zapisane w pamięci komputera. Jednym z najpopularniejszych i najczęściej używanych standardów Unicode jest UTF-8.

Kodowanie i dekodowanie punktów kodowych

Aby sprawdzić jaki jest kod w ASCII/UNICODE określonego znaku, możesz użyć funkcji o nazwie ord()

Jeżeli znasz punkt kodowy i chcesz uzyskać wartość to Python udostępnia do tego funkcję chr()

```
print(ord("A"))
print(ord(" "))
print(ord("z"))
print(ord("."))
print(chr(90))
print(chr(65))
print(chr(32))
if chr(ord("A")) == "A":
    print("Prawda")
if ord("A") == 65:
    print("Prawda")
```

```
65
32
122
46
Z
A

Prawda
Prawda
```

Łańcuch znaków (string)

Do elementu łańcucha znaków możemy odwoływać się za pomocą indeksów

```
lancuch = "ABC test"
print(lancuch[0]) # A
print(lancuch[5]) # e
```

```
A
e
```

Po łańcuchu znaków można również się iterować pętlą for

```
lancuch = "test ABC"
for i in lancuch:
    print(i, end='')
```

```
test ABC
```

Łańcuch znaków (string)

Wykorzystanie wycinków z ciągu znaków (string)

```
lancuch = "test ABCD"
print(lancuch[5:8]) # ABC
```

```
ABC
```

Wykorzystanie operatorów in i not in

```
tekst = "test ABCD"
print("A" in tekst) # True
print("B" in tekst) # True
print("B" not in tekst) # False
```

```
True
True
False
```

Łańcuch znaków (string)

Łańcuchy znaków nie są modyfikowalne. Nie można używać instrukcji del do usuwania elementów z łańcuch. Nie można używać funkcji append() i insert() do dodawania elementów do ciągu znaków. Do modyfikowania łańcucha znaków należy wykorzystywać operację jego nadpisywania

```
tekst = "test ABCD"
tekst = "nowy ciąg"
tekst = tekst + " znaków" # tekst += " znaków"
print(tekst)
```

```
nowy ciąg znaków
```

Łańcuch znaków (string)

Można używać funkcji min() i max() zwracają one wynik na podstawie punktu kodowego znaku.

```
tekst = "test ABCD"
print(min(tekst)) # " "
print(max(tekst)) # "t"
```

```
 
t
```

Metody index() i find()

Metoda index() służy do pobrania numeru indeksu pierwszego wystąpienia szukanego fragmentu

```
tekst = "test ABCD"
print(tekst.index("t")) # 0
print(tekst.index("A")) # 5
```

```
0
5
```

Metoda find() działa podobnie, tylko w przypadku nie znalezienia pasującego elementu zwróci -1 zamiast błędu. Metoda find() potrafi również przeszukiwać łańcuch od wskazanego indeksu.

```
tekst = "test ABCD"
print(tekst.find("t")) # 0
print(tekst.find("z")) # -1
print(tekst.find("B", 1)) # 6
```

```
0
-1
6
```

Metody capitalize(), lower(), upper(), title()

- Metoda capitalize() zamienia podany łańcuch w taki sposób, że jeżeli pierwszy znak jest literą to zamienia go na duża literę a pozostałe znaki na małe litery
- Metoda lower() zamienia wszystkie znaki w łańcuchu na małe litery.
- Metoda upper() zamienia wszystkie znaki w łańcuchu na wielkie litery.
- Metoda title() zamienia wszystkie pierwsze znaki w każdym słowie. występującym w łańcuchu na wielkie litery litery.

```
tekst = "test ABCD"
print(tekst.capitalize())
print(tekst.upper())
print(tekst.lower())
print(tekst.title())
```

```
Test abcd
TEST ABCD
test abcd
Test Abcd
```

Metoda center()

Metoda center() próbuje podany łańcuch wyświetlić na środku łańcucha o podanej szerokości.

```
tekst = "test ABCD"
print("[" + tekst.center(11) + "]") # [ test ABCD ]
print("[" + tekst.center(11, "8") + "]") # [8test ABCD8]
```

```
[ test ABCD ]
[8test ABCD8]
```

Metody endswith(), startswith()

Metoda endswith() weryfikuje, czy łańcuch kończy się podanym argumentem.

Metoda sprawdzająca czy łańcuch zaczyna się od podanego argumentu to startswith.

```
tekst = "test ABCD"
print(tekst.endswith("CD")) # True
print(tekst.endswith("te")) # False
```

```
True
False
```

Metody isalnum(), isalpha(), isdigit()

- Metoda isalnum() sprawdza czy wszystkie znaki są literami albo liczbami.
- Metoda isalpha() jest bardziej specyficzna i sprawdza czy znaki są literami.
- Metoda isdigit() sprawdza czy znaki są liczbami.

```
tekst = "test ABCD"
print(tekst.isalnum()) # True
print("test123".isalnum()) # True
print("".isalnum()) # False
print("56".isalnum()) # False
```

```
False
True
False
True
```

Metody islower(), isupper(), isspace()

- Metoda islower() sprawdza czy wszystkie znaki są małymi literami.
- Metoda isupper() sprawdza czy wszystkie znaki są dużymi literami.
- Metoda isspace() sprawdza czy wszystkie znaki są spacjami.

```
tekst = "test ABCD"
print(tekst.islower()) # False
print(tekst.islower()) # False
print("TEST".isupper()) # True
print(" ".isspace()) # True
print("\n".isspace()) # True
```

```
False
False
True
True
True
```

Metoda replace()

Metoda replace() zamienia wszystkie wystąpienia łańcucha podanego jako pierwszy argument na łańcuch podany jako drugi argument.

```
tekst = "test ABCD"
print(tekst.replace("test", ""))

tekst = "test ABCD"
print(tekst.replace("test", "jakiś"))
```

```
 ABCD
jakiś ABCD
```

Metoda split()

Metoda split() dzieli łańcuch znaków względem podanego argumentu. Wynikiem działania funkcji jest lista pod łańcuchów podzielonych względem podanego ciągu.

```
tekst = "test ABCD Adam"
lista = tekst.split(" ")
print(lista)
```

```
['test', 'ABCD', 'Adam']
```

Metoda strip()

Metoda strip() usuwa wszystkie spacje znajdujące się na początku i końcu łańcucha. Metody lstrip() i rstrip() usuwają odpowiednio spacje z początku i z końca podanego łańcucha.

```
tekst = "  test ABCD Adam "
print(tekst)
print(tekst.strip())
```

```
  test ABCD Adam 
test ABCD Adam
```

Porównywanie łańcuchów znaków

Język Python potrafi porównywać łańcuchy znaków za pomocą operatorów porównania == != > >= < <=

Język Python tak naprawdę porównuje punkty kodowe na poszczególnych pozycjach i na ich podstawie zwraca wynik.

```
tekst = "test"
print(tekst == 'test') # True
print('Tekst' != 'tekst') # True
print('tekst' > 'Tekst') # True
print('teks' > 'tekst') # False
```

```
True
True
True
False
```

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 20 minutowe

Proszę stworzyć program program sprawdzający czy podany przez użytkownik ciąg

- znaków zawiera
- małe litery
- duże litery
- cyfry
- jeden ze znaków $#@
- minimum 5 znaków
- maksimum 10 znaków

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 40 minutowe

Proszę stworzyć program który będzie działał na zasadzie Szyfru Cezara

- Program powinien spytać użytkownika o czynność którą chce wykonać - kodowanie / odkodowanie
- Spytać o wiadomość
- Spytać o punkt przesunięcia, jeżeli użytkownik nic nie poda to przyjmie domyślną wartość - 3
- Wyświetlić zakodowany / odkodowany tekst

Wyjątki

Wyjątek (ang. exception) – to mechanizm przepływu sterowania używany w procesorach oraz współczesnych językach programowania. W celu obsługi zdarzeń wyjątkowych, a w szczególności błędów, których wystąpienie zmienia prawidłowy przebieg wykonywania programu. W momencie zajścia niespodziewanego zdarzenia generowany jest wyjątek, który musi zostać obsłużony poprzez zapamiętanie bieżącego stanu programu i przejście do procedury jego obsługi. W niektórych sytuacjach po obsłużeniu wyjątku można powrócić do wykonywania przerwanego kodu, korzystając z zapamiętanych informacji stanu. Przykładowo obsługa błędu braku strony pamięci polega najczęściej na pobraniu brakującej strony z pliku wymiany, co umożliwia kontynuowanie pracy programu, natomiast błąd dzielenia przez zero powoduje, że wykonywanie dalszych obliczeń nie ma sensu i musi zostać definitywnie przerwane.

Wyjątki

Wyjątki to nieprzewidziane, niemożliwe do wykonania operacje w trakcie działania programu.

Wyjątkiem może być:

- Próba dzielenia przez 0,
- Konwersja niepoprawnego ciągu na wartość liczbową.

Jeżeli program zgłosi wyjątek i nie jest on obsłużony następuje natychmiastowe zakończenie działania programu.

Wyjątki

Python dostarcza programistom możliwość obsługi niepoprawnego wywołania kodu w trakcie działania programu. Do obsługi wyjątków służy blok:

try ... except ...

```
a = 1
b = 0

try:
    print(a / b)
except:
    print("próba dzilenia przez 0!")
    print("koniec działania programu")
```

```
próba dzilenia przez 0!
koniec działania programu
```

Obsługa wyjątków w Python

Blok kodu narażony na błędne wykonanie może zwrócić więcej niż jeden wyjątek można to obsłużyć wieloma blokami `except`. Jeżeli wyjątek zostaje zwrócony to wykonany jest pierwszy pasujący blok `except`

```
a = 1
b = 0
try:
    print(a / b)
except ZeroDivisionError:
    print("próba dzilenia przez 0!")
except:
    print("nieznany wyjątek")
    print("koniec działania programu")
```

```
próba dzilenia przez 0!
```

Wyjątki - hierarchia

Wyjątki w języku Python mają swoją hierarchię, najbardziej ogólnym wyjątkiem jest BaseException.
Wszystkie wyjątki (bardziej szczegółowe) dziedziczą z tego głównego wyjątku
Np. ZeroDivisionError < ArithmeticError < Exception < BaseException

```
try:
    raise ArithmeticError("błąd")
except ZeroDivisionError:
    print("ZeroDivisionError")
except ArithmeticError:
    print("ArithmeticError")
except Exception:
    print("Exception")
except BaseException:
    print("BaseException")
    print("koniec działania programu")
```

```
ArithmeticError
```

Link do spisu wyjątków Python:
https://docs.python.org/3/library/exceptions.html

Wyjątki - hierarchia

Python pozwala obsłużyć wiele rodzajów wyjątków jednocześnie.

Wyjątki które chcemy obsłużyć jednym blokiem wypisujemy w nawiasie po przecinku.

```
try:
    raise ArithmeticError("błąd")
except ZeroDivisionError:
    print("ZeroDivisionError")
except (ArithmeticError, ValueError):
    print("ArithmeticError")
except Exception:
    print("Exception")
except BaseException:
    print("BaseException")

print("koniec programu")
```

Python pozwala na zgłaszanie wyjątków bezpośrednio z kodu a nie dopiero na skutek jego wystąpienia.

Aby zgłosić wyjątek należy użyć polecenia raise.

```
ArithmeticError
koniec działania programu
```

Asercje

Instrukcja assert sprawdza poprawność wyrażenia i jeżeli nie zwraca ono wartości prawdziwej (True, liczba różna od 0, niepusty ciąg znaków, nie None) wówczas wywoływany jest wyjątek AssertionError.

```
zm = 0

try:
    assert zm > 0
except AssertionError:
    print("Błąd w Asercji!")
print("koniec programu")
```

```
Błąd w Asercji!
koniec programu
```

Wyjątki i else

Można wykorzystać konstrukcję try ... except ... else

Blok kodu z else wykona się wtedy i tylko wtedy gdy blok try nie zwróci żadnego wyjątku.

```
zm = 1
try:
    print(1/zm)
except:
    print("Nład w kodzie!")
else:
    print("Nie było błędu w kodzie")
print("Koniec programu")
```

```
Nie było błędy w kodzie
Koniec programu
```

```
zm = 0
try:
    print(1/zm)
except:
    print("Błąd w kodzie!")
else:
    print("Nie było błędu w kodzie")
print("Koniec programu")
```

```
Błąd w kodzie!
Koniec programu
```

Wyjątki i finally

Python udostępnia również blok finally który umieszczony jest na końcu bloku try ... except ... i służy do wykonania kodu bez względu na to czy kod wykonał się poprawnie czy zwrócił błąd. Blok finally może istnieć również z blokiem else.

```
zm = 1
try:
    print(1/zm)
except:
    print("Błąd kodu!")
finally:
    print("Koniec try")
print("Koniec programu")
```

```
1.0
Koniec try
Koniec programu
```

Wyświetlanie wyjątków jako event

Python udostępnia możliwość pobrania i wyświetlenia większej ilości informacji o błędzie.

```
zm = 0

try:
    print(1/zm)
except Exception as e:
    print(e)
print("Koniec programu")
```

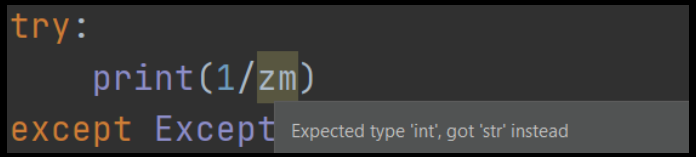
```
division by zero
Koniec programu
```

Wyświetlanie wyjątków jako event

Python udostępnia możliwość pobrania i wyświetlenia większej ilości informacji o błędzie.

```
zm = "0"
try:
    print(1/zm)
except Exception as e:
    print(e)
print("Koniec programu")
```

Środowisko (IDE) może wskazać błąd:




```
unsupported operand type(s) for /: 'int' and 'str'
Koniec programu
```

Klasy i obiekt (Programowanie obiektowe)

```
class MojaKlasa:
    pass
```

Klasa to forma (definicja) dla obiektów.

Definicja obejmuje dopuszczalny stan obiektów oraz ich zachowania.

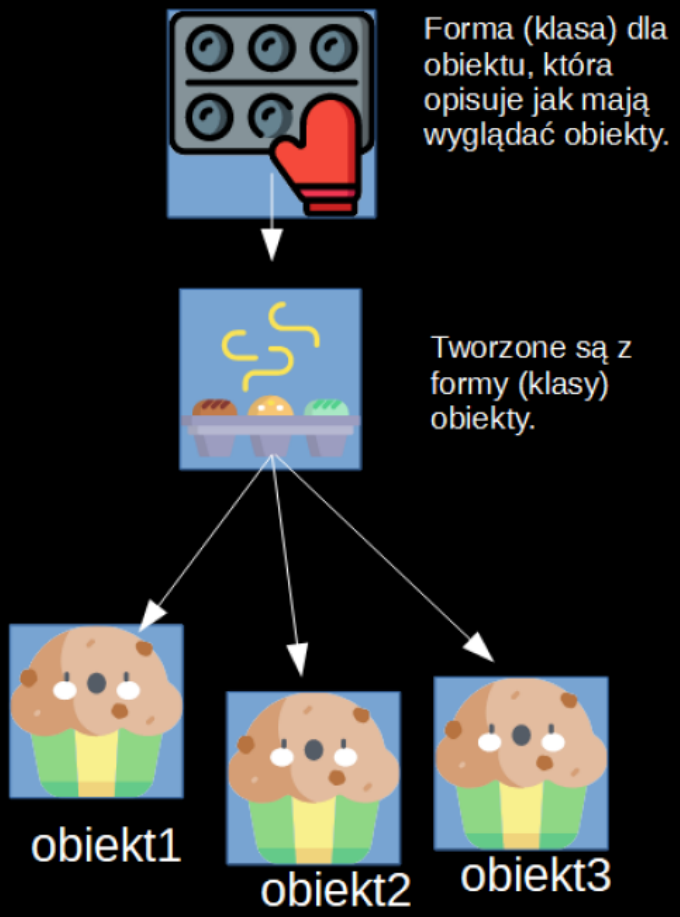
```
zm = MojaKlasa()
print(zm)
print(type(zm))
```

Obiekt to element programu, który **został stworzony na podstawie klasy**.

Obiekt utworzony na podstawie definicji klasy nazywa się jej instancją

```
<__main__.MojaKlasa object at 0x0000016568933C8>
<class '__main__.MojaKlasa'>
```

Klasy (Programowanie obiektowe)



Konstruktor klasy

Jest to funkcja znajdująca się wewnątrz klasy, która **wykonywana jest podczas tworzenia obiektów danej klasy, dodatkowo służy do nadania początkowych wartości własnością klasy**. Konstruktor musi posiadać co najmniej jeden parametr self (lub inna dowolna nazwa) który jest odwołaniem się do obiektu i pozwala dodawać do niego elementy.

```
class MojaKlasa:
    def __init__(self):
        self.zm1 = 0
        self.zm2 = "coś"

obiekt = MojaKlasa()
print(obiekt.zm1)
print(obiekt.zm2)
```

```
0
coś
```

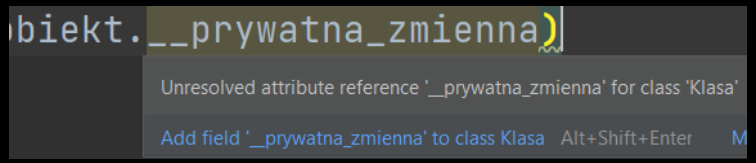
Pole prywatne klasy

W Python można definiować pola prywatne dla klasy, czyli takie które dostępne są wyłącznie wewnątrz klasy. W celu zdefiniowania elementu klasy jako prywatnego należy użyć __ (podwójnego znaku podkreślenia).

```
class Klasa:
    def __init__(self):
        self.__prywatna_zmienna = 0

obiekt = Klasa()
print(obiekt.__prywatna_zmienna)
```

Srodowisko programistyczne (IDE) może wskazać błąd przed wykonaniem:




```
    print(obiekt.__prywatna_zmienna) # AttributeError
AttributeError: 'Klasa' object has no attribute '__prywatna_zmienna'
```

Metody klasy

Metoda to funkcja zdefiniowana wewnątrz klasy mająca dostęp do wszystkich składników klasy

```
class Klasa:
    def __init__(self):
        self.__prywatna_zmienna = 55
    def get_prywatna_zmienna(self):
        return self.__prywatna_zmienna

obiekt = Klasa()
print(obiekt.get_prywatna_zmienna())
```

Jeżeli wykonamy metodę get, to zwróci ona zawartość pola prywatnego.

```
55
```

Zmienne w klasie

Zmienne klasy (zwane też statycznymi) są to zmienne które nie potrzebują instancji danej klasy aby można było je odczytać oraz są współdzielone przez wszystkie instancje (wystąpienia) danej klasy.

```
class Klasa:
    licznik = 0
    def __init__(self):
        Klasa.licznik += 1

obiekt = Klasa()
obiekt2 = Klasa()
print(Klasa.licznik)
```

```
2
```

Atrybuty klasy hasattr()

Python jest językiem dynamicznym i w dowolnym momencie można dodać nową wartość do obiektu, aby sprawdzić czy obiekt posiada interesujący atrybut (własność) można użyć funkcji hasattr()

```
class Klasa:
    def __init__(self):
        self.imie = "Bartosz"

obiekt = Klasa()
print(hasattr(obiekt, 'imie'))
print(hasattr(obiekt, 'nazwisko'))
```

```
True
False
```

Klasy (Programowanie obiektowe)

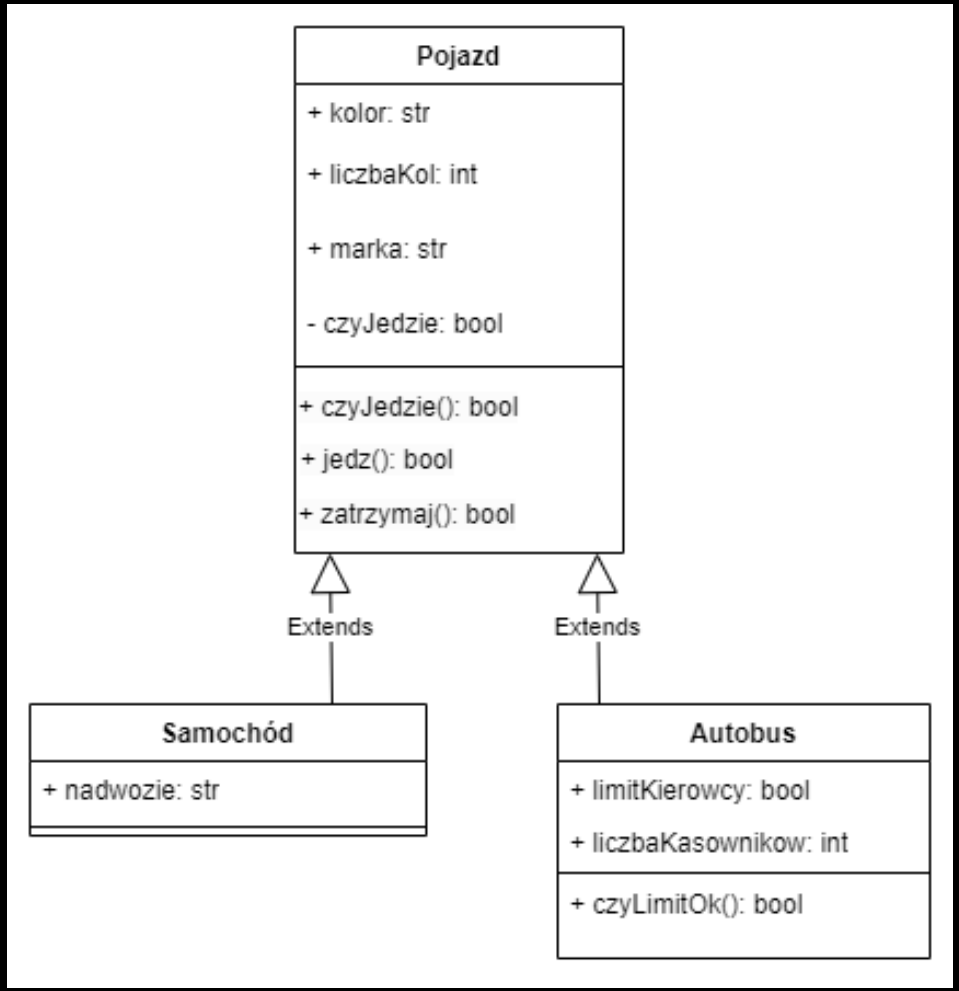
Python dysponuje metodami do pobierania atrybutów klasy getattr() i ustawiania atrybutów klasy setattr()

```
class Klasa:
    def __init__(self):
        self.imie = "Ala"

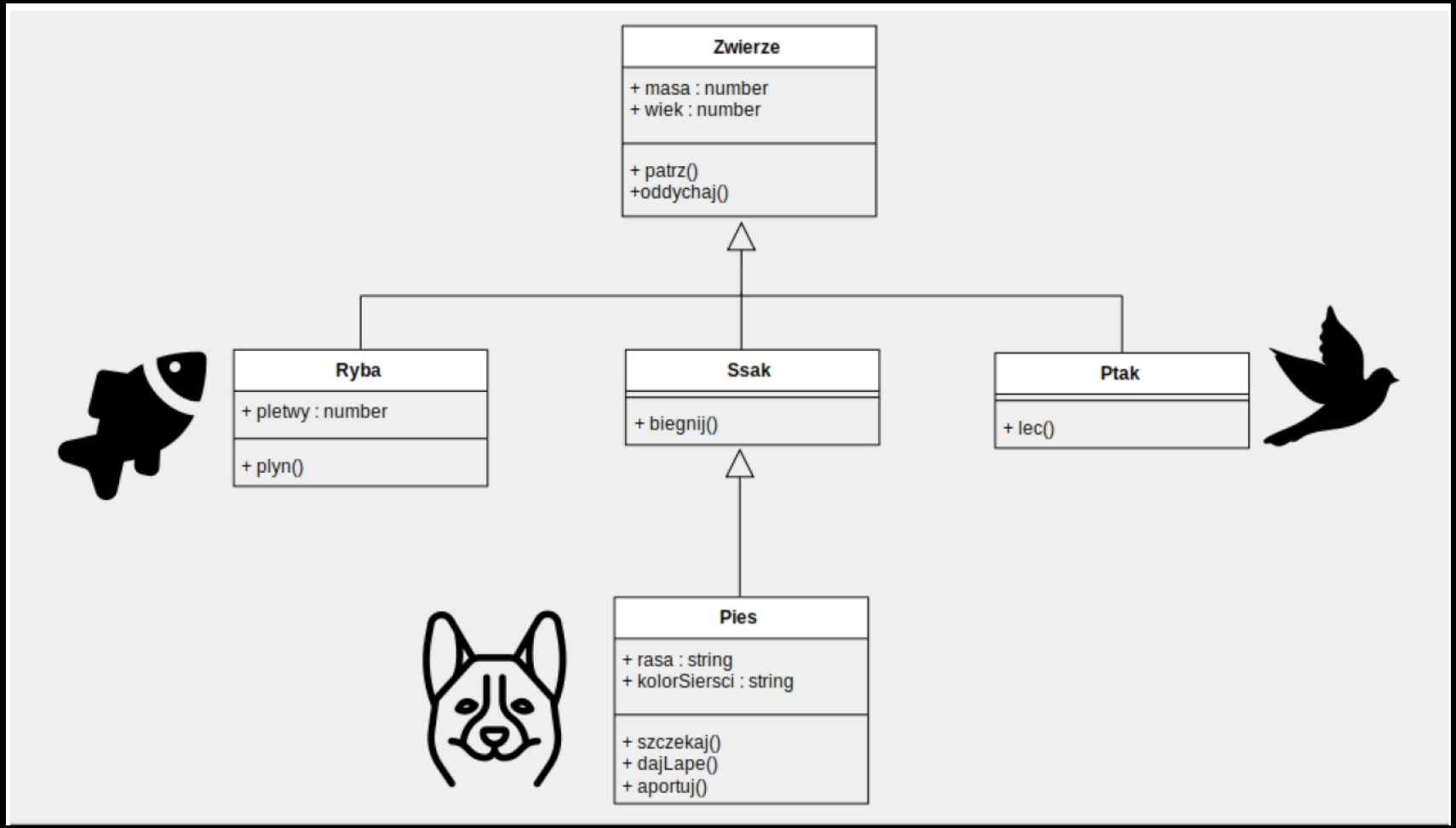
obiekt = Klasa()
print(getattr(obiekt, "imie"))
setattr(obiekt, 'imie', "Ada")
print(getattr(obiekt, "imie"))
```

```
Ala
Ada
```

Dziedziczenie (Programowanie obiektowe)



Dziedziczenie

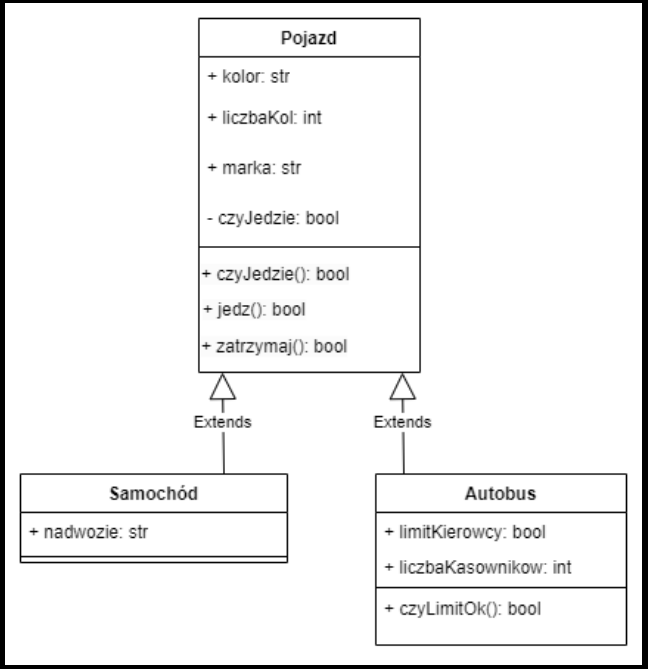


Dziedziczenie

Klasy możemy układać w hierarchię i tworzyć nowe definicje klas na podstawie już istniejących. Aby klasa dziedziczyła po innej klasie należy przy definicji nowej klasy w nawiasach wpisać nazwę klasy którą rozszerzamy.

```
class Pojazd:
    def __init__(self, kolor, liczbaKol, marka):
        self.kolor = kolor
        self.liczbaKol = liczbaKol
        self.marka = marka
        self.__czyJedzie = False
    def czyJedzie(self):
        return self.__czyJedzie
    def jedz(self):
        self.__czyJedzie = True
    def zatrzymaj(self):
        self.__czyJedzie = False

class Samochod(Pojazd):
    def __init__(self, kolor, liczbaKol, marka, nadwozie):
        super().__init__(kolor, liczbaKol, marka)
        self.nadwozie = nadwozie
```



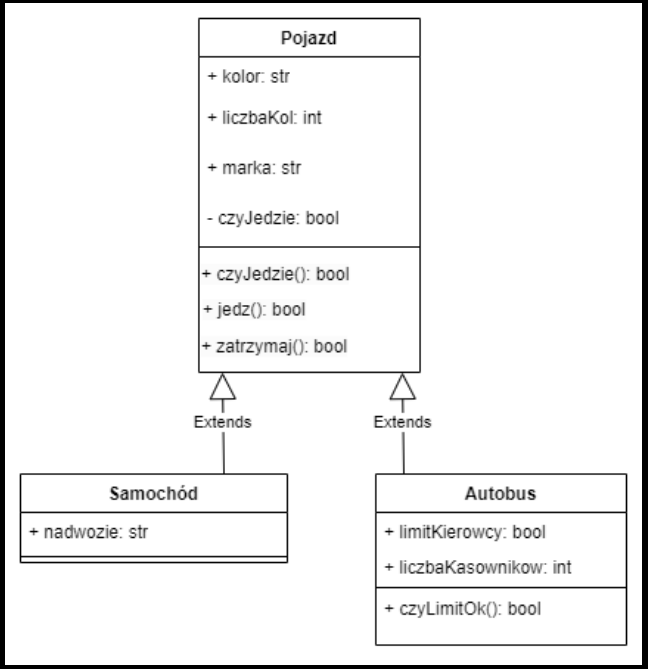
c. d. n. (za mało miejsca)

Dziedziczenie (cz. 1)

Klasy możemy układać w hierarchię i tworzyć nowe definicje klas na podstawie już istniejących. Aby klasa dziedziczyła po innej klasie należy przy definicji nowej klasy w nawiasach wpisać nazwę klasy którą rozszerzamy.

```
class Pojazd:
    def __init__(self, kolor, liczbaKol, marka):
        self.kolor = kolor
        self.liczbaKol = liczbaKol
        self.marka = marka
        self.__czyJedzie = False
    def czyJedzie(self):
        return self.__czyJedzie
    def jedz(self):
        self.__czyJedzie = True
    def zatrzymaj(self):
        self.__czyJedzie = False

class Samochod(Pojazd):
    def __init__(self, kolor, liczbaKol, marka, nadwozie):
        super().__init__(kolor, liczbaKol, marka)
        self.nadwozie = nadwozie
```



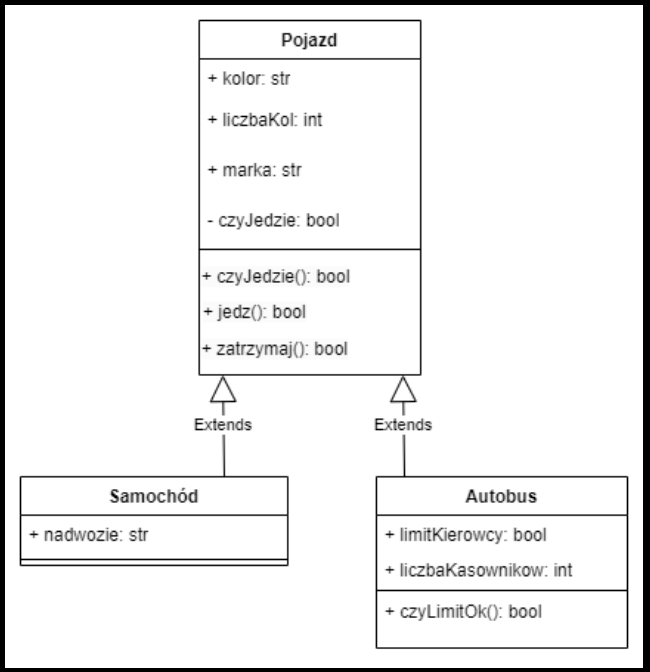
c. d. n. (za mało miejsca)

Dziedziczenie (cz. 2)

Tworzenie instancji klasy:

```
obiekt = Samochod("Czarny", 4, "Toyota", "SUV")
print(obiekt.kolor)
print(obiekt.nadwozie)
print(obiekt.marka)
print(obiekt.liczbaKol)
```

```
Czarny
SUV
Toyota
4
```



Instancja – pojedyncze wystąpienie niezależnego kodu zgodnego z danym wzorcem.

Dziedziczenie – kolejny przykład

```
class Klasa:
    def __init__(self):
        self.zmienna = "test"

class Klasa2(Klasa):
    def get_zmienna(self):
        return self.zmienna

obiekt = Klasa2()
print(obiekt.get_zmienna())
```

```
test
```

Weryfikacja dziedziczenia

W Python możliwe jest sprawdzenie czy jedna klasa dziedziczy z innej klasy (jest jej podklasą) służy do tego instrukcja issubclass()

```
class Klasa:
    def __init__(self):
        self.zmienna = "test"
class Klasa2(Klasa):
    def get_zmienna(self):
        return self.zmienna

obiekt = Klasa2()
print(issubclass(Klasa2, Klasa))
print(issubclass(Klasa, Klasa2))
```

```
True
False
```

Czy obiekt jest instancją klasy

W Python możliwe jest zweryfikowanie czy obiekt jest instancją danej klasy, służy do tego instrukcja **isinstance()**

```
class Klasa:
    pass
class Klasa2(Klasa):
    pass

obiekt = Klasa()
print(isinstance(obiekt, Klasa))
obiekt2 = Klasa2()
print(isinstance(obiekt2, Klasa2))
print(isinstance(obiekt2, Klasa))
```

```
True
True
True
```

Sprawdzanie czy elementy są identyczne

Można wykorzystać operator **is** , który pozwala sprawdzić czy dwa obiekty są identyczne.

```
class Klasa:
    pass

obiekt = Klasa()
obiekt2 = Klasa()
obiekt3 = obiekt
print(obiekt is obiekt2)
print(obiekt is obiekt3)
```

```
False
True
```

Wyświetlanie zawartości obiektów

Python ma zaimplementowaną domyślną metodę która wyświetla zawartość obiektu, możemy zmienić sposób wyświetlania obiektu danej klasy poprzez nadpisanie metody __str__

```
class Klasa:
    pass

class Klasa2:
    def __str__(self):
        return "Klasa2"
obiekt = Klasa()
obiekt2 = Klasa2()
print(obiekt)
print(obiekt2)
```

```
<__main__.Klasa object at 0x00000290660B9D88>
Klasa2
```

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 30 minutowe

Proszę napisać program, który:

- posiada klasę Person (pola: imię, nazwisko, wiek, płeć, przeciążoną metodę __str__)
- posiada klasę Student (dziedziczy po klasie Person, dodatkowo zawiera pole z listą ocen i metodę do wyliczania średniej)
- posiada klasę Pracownik (dziedziczy po klasie Person, dodatkowo zawiera pole z informacją o zarobkach)

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 30 minutowe

Należy stworzyć klasę, która omawia problematykę Hotelu. Na podstawie takiej klasy, powinniśmy móc stworzyć wiele obiektów hoteli. Czyli klasa jest wzorem opisującym Hotel.

- kilka właściwości klasy,
- kilka metod klasy,konstruktor klasy.
- Na podstawie klasy proszę stworzyć wiele różnych obiektów Hoteli.

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 40 minutowe

Proszę stworzyć prosty diagram klas UML

- Proszę wymyślić problematykę programu, czyli jakie klasy utworzyć oraz która ma dziedziczyć po drugiej.
- Proszę zaprojektować diagram w poniższym narzędziu online do rysowania diagramów.
- Na podstawie diagramu proszę utworzyć odpowiednie klasy i wykorzystać mechanizm dziedziczenia.

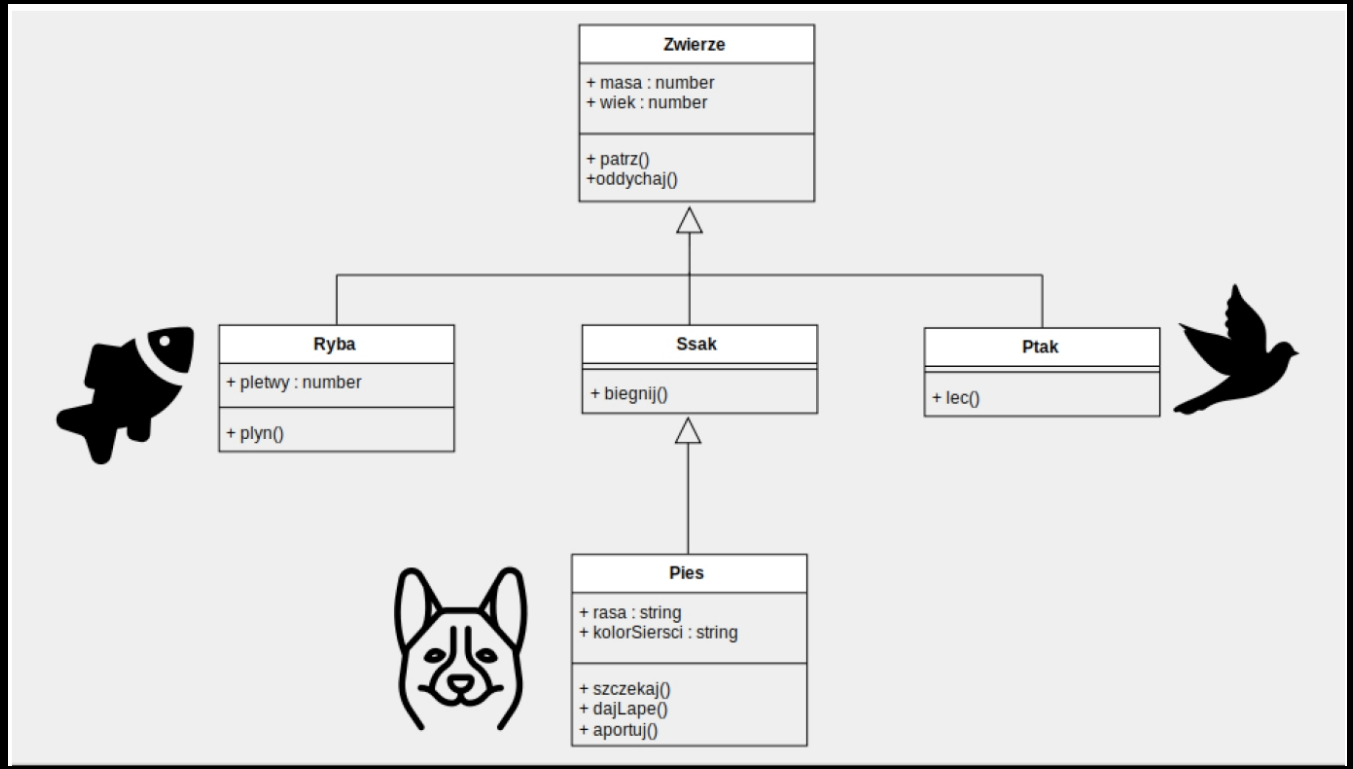
Należy wykorzystać narzędzie online: *https://app.diagrams.net/*

Ćwiczenie

Eksperymenty z użyciem łańcuchów znaków

ćwiczenie 20 minutowe

Proszę na podstawie poniższego diagramu klas UML napisać odpowiednie klasy w języku Python.



Własne wyjątki – użycie klasy

Python pozwala definiować własne wyjątki za pomocą mechanizmu dziedziczenia.

Klasa może dziedziczyć z klasy Exception

```
class wlasnyWyjatek(Exception):
    pass
try:
    raise wlasnyWyjatek()
except wlasnyWyjatek:
    print("mój wyjątek")
except:
    print("domślny wyjątek")
```

Własne wyjątki – użycie klasy (przykład)

```
class InvalidAgeError(Exception):
    def __init__(self, age):
        super().__init__(f'Bad age! {age}')

def getDataFromUser():
    age = int(input("Podaj swój wiek: [zatwierdź enterem]"))
    if age < 0 or age >= 130:
        raise InvalidAgeError(age)
    else:
        print(f'Twój wiek to: {age}')

while True:
    getDataFromUser()
```

Generatory

Generatory to funkcje które zapamiętują stan pomiędzy wywołaniami i zwracają kolejną wartość.

Generator to funkcja która zawiera wystąpienie słowa yield zamiast return.

```
def fn(n):
    for i in range(n):
        yield i

elem = fn(5)
print(elem)

for i in elem:
    print(i)
```

```
<generator object fn at 0x00000157562990C8>
0
1
2
3
4
```

Generatory

```
def fn(n):
    for i in range(n):
        yield i

elem = fn(5)
print(list(elem))
print(list(fn(6)))
print(list(fn(6)))

for i in elem:
    print(i)
```

```
[0, 1, 2, 3, 4]
[0, 1, 2, 3, 4, 5]
[0, 1, 2, 3, 4, 5]
```

Generatory

```python
def fn(n):
    for i in range(n):
        yield i

print(list(fn(6)))
print(list(fn(9)))

for i in list(fn(4)):
    print(i)
```

```
[0, 1, 2, 3, 4, 5]
[0, 1, 2, 3, 4, 5, 6, 7, 8]
0
1
2
3
```

Podsumowując, funkcja fn(n) zwraca sekwencję liczb od 0 do n-1 jako generator, który generuje wartości na żądanie podczas iteracji. Funkcja ta może być używana w różnych kontekstach, na przykład do tworzenia skutecznych i wydajnych sekwencji liczb w przypadkach, gdy nie chcemy przechowywać całej sekwencji w pamięci na raz.

Klasy (Programowanie obiektowe)

Do zakończenia działania iteratora możemy też wykorzystać instrukcję StopIteration

```
def fn(n):
    i = 0
    while True:
        if i == n:
            return StopIteration
        yield i
        i += 1
elem = fn(5)
print(list(elem))
```

```
[0, 1, 2, 3, 4]
```